DATA ENGINEERING FRESHER INTERVIEW STUDY GUIDE

Databricks • Spark • Delta Lake • Azure • SQL • Data Warehousing • Python

Prepared as a beginner-friendly revision handbook from the interview question bank shared in chat. Each topic follows: simple meaning, precise definition, practical implication, example, and interview-ready wording.

Best answering method

Technical: Definition → Purpose → Example
Scenario: Investigate → Cause → Fix → Validate

How to Use This Guide

- Learn the simple meaning first.
- Memorize only the short interview answer, not the whole page.
- Practice each answer aloud in 30-60 seconds.
- For project and training answers, mention only work you actually completed.
- For production scenarios, do not claim unsupported access. Explain safe investigation, escalation, approval, rerun, and validation.

Priority Map

Priority	Topics
Tier 1 - Must know	Databricks, Spark architecture, Driver/Executor, DataFrame, RDD, Catalyst, transformations/actions, partitioning, broadcast join, Medallion, Delta vs Parquet, ACID, MERGE, failures, SQL joins, windows.
Tier 2 - Important	ADF, ADLS vs Blob, Key Vault, SAS, keys, normalization, fact/dimension, star/snowflake, SCD, CTE, subqueries, indexes, Python collections and OOP.
Tier 3 - Revise	Views, materialized views, measure types, dimension types, decorators, recursive CTE, database connectors.

Part 1 - DataFrame, RDD and Dataset

1. What is a DataFrame?

Simple meaning: A DataFrame looks like an Excel or SQL table, but its rows can be distributed across many machines for parallel processing.

Definition: A Spark DataFrame is a distributed collection of structured data organized into named columns. Spark SQL can optimize DataFrame operations using the Catalyst Optimizer.

```
df = spark.read.csv("/path/employees.csv", header=True, inferSchema=True)

df.select("name", "salary").show()
```

How to say it: A DataFrame is a distributed collection of structured data organized into named columns. I prefer it for structured ETL because it provides a schema, high-level APIs and Catalyst optimization.

2. RDD vs DataFrame vs Dataset

Feature	RDD	DataFrame	Dataset
Meaning	Low-level distributed collection	Distributed rows and named columns	Strongly typed distributed collection
Schema	No table-like schema	Yes	Yes
Optimization	Limited	Catalyst/Spark SQL	Catalyst/Spark SQL
Common API	map, filter, reduce	select, filter, groupBy, join	Typed Scala/Java operations
PySpark	Available	Preferred for ETL	Dataset API is mainly Scala/Java

How to say it: RDD is Spark's low-level distributed collection. DataFrame represents structured rows and columns and benefits from Spark SQL optimization. Dataset adds compile-time type safety mainly in Scala and Java. In PySpark ETL, I generally use DataFrames.

3. Why prefer DataFrames over RDDs?

- Schema and named columns
- Higher-level, readable APIs
- Catalyst query optimization
- Interoperability with SQL
- Efficient structured ETL

How to say it: For structured data, I prefer DataFrames because they provide schema, SQL-like operations and optimizer-aware execution. RDDs are useful when low-level control is required, but they usually need more manual code.

4. Can DataFrames be joined?

Yes. Spark supports inner, left, right, full, left semi, left anti and cross joins.

```
result = customers_df.join(orders_df, "customer_id", "inner")
```

5. Join a 5 GB DataFrame with a 50 MB DataFrame

A broadcast join may be suitable if the smaller DataFrame fits safely in executor memory. Spark sends the small side to executors, reducing the need to shuffle the large side.

```
from pyspark.sql.functions import broadcast

result = large_df.join(broadcast(small_df), "customer_id")
```

How to say it: Since one DataFrame is much smaller, I would consider a broadcast join if it fits safely in executor memory. I would verify the execution plan and memory instead of broadcasting only because the source file is 50 MB.

Part 2 - Spark Fundamentals

6. What is Apache Spark?

Spark is a distributed processing engine. It divides large data into partitions and processes them in parallel on executors.

How to say it: Apache Spark is a distributed data-processing engine designed for large-scale workloads. It divides data into partitions and processes those partitions in parallel across executors.

7. Spark Architecture

Application → Driver → Cluster Manager → Executors → Tasks → Partitions

Component	Role
Driver	Creates SparkSession, builds plans, schedules work and coordinates the application.
Cluster manager	Allocates CPU and memory resources.
Executor	Runs tasks, processes partitions and may cache data.
Task	Small unit of work executed for a partition.
Partition	A logical chunk of distributed data.

How to say it: The driver coordinates the Spark application and prepares execution. The cluster manager allocates resources. Executors process partitions by running tasks. Lazy transformations execute when an action triggers a job.

8. Why use clusters?

Clusters provide distributed CPU and memory. This allows large datasets to be processed in parallel and lets resources scale according to workload size.

9. Transformations and Actions

Transformations create a new DataFrame and are usually lazy: select, filter, withColumn, groupBy and join. Actions trigger execution: show, count, collect, write and save.

```
filtered = df.filter(df.salary > 50000)  # transformation
filtered.show()                          # action
```

10. Catalyst Optimizer

Catalyst is Spark SQL's query optimization framework. It analyzes the logical plan, applies optimization rules and helps choose a physical execution plan.

How to say it: Catalyst Optimizer is Spark SQL's query optimization framework. It analyzes and optimizes logical plans before Spark executes the selected physical plan.

11. Cache vs Persist

Caching or persisting helps when the same expensive DataFrame is reused. Without it, Spark may recompute the lineage. Unnecessary caching consumes executor memory.

cache()	**persist()**
Convenient persistence using Spark's default cache storage level.	Allows an explicit storage level such as memory/disk combinations.

How to say it: I cache only a reused, expensive intermediate DataFrame. If I do not cache it, Spark can recompute it from lineage. Caching everything is harmful because it consumes executor memory.

12. repartition vs coalesce

Factor	**repartition**	**coalesce**
Purpose	Redistribute data	Usually reduce partitions
Shuffle	Full shuffle	Usually avoids full shuffle
Increase partitions	Yes	Generally not used
Balance	Can improve balance	May preserve imbalance
Use	Before wide operations or to rebalance	Reduce output files/partitions

How to say it: Repartition performs a shuffle and redistributes data, so it can increase or decrease partitions and improve balance. Coalesce generally reduces partitions with less data movement.

Part 3 - Databricks and Medallion Architecture

13. What is Databricks?

Databricks is a cloud-based data and AI platform used to ingest, process, transform, govern and analyze data. It integrates Spark, notebooks, compute, Delta Lake, SQL, workflows and ML capabilities.

> *How to say it: Databricks is a cloud-based data and AI platform used for large-scale data processing with Apache Spark. It provides notebooks, scalable compute, Delta Lake, workflows, SQL analytics and governance capabilities.*

14. Databricks Architecture

Users → Notebooks / SQL / Jobs → Databricks Compute → Spark Driver and Executors → Cloud Storage / Delta Tables

Do not confuse Databricks platform architecture with Medallion data architecture. Spark driver/executor architecture explains processing; Bronze/Silver/Gold explains how data is organized.

15. What is a Notebook?

A notebook is an interactive workspace containing executable cells. It can support languages such as Python, SQL, Scala and R depending on the runtime and configuration.

> *How to say it: A Databricks notebook is an interactive development environment where we write and execute code in cells, inspect results and collaborate on data-processing tasks.*

16. Medallion Architecture

Layer	Meaning	Typical work
Bronze	Raw or minimally processed ingested data	Capture source data and ingestion metadata.
Silver	Cleaned, validated and standardized data	Deduplicate, handle nulls, cast types, validate and join.
Gold	Business-ready data	Aggregations, KPIs, reporting and analytics tables.

> *How to say it: Medallion architecture organizes data into Bronze, Silver and Gold. Bronze stores raw ingested data, Silver contains cleaned and validated data, and Gold provides aggregated or business-ready data for analytics.*

17. Gold-layer consumers

- Databricks SQL
- Power BI or other BI tools
- Dashboards and reporting
- Analytics or ML applications

Part 4 - Delta Lake

18. Delta table vs Parquet files

Feature	Parquet	Delta
Meaning	Columnar file format	Table format/protocol using data files plus transaction log
Transaction log	No Delta table log	_delta_log

ACID	No Delta table-level ACID	Supported
UPDATE/DELETE/MERGE	Manual rewriting/logic	Supported table operations
History/time travel	Not inherent	Supported while history/files are retained
Schema	Reader/writer dependent	Schema enforcement and controlled evolution

How to say it: Parquet is a columnar file format. Delta Lake adds a transaction log and table-management capabilities over data files, including ACID transactions, history, schema enforcement and MERGE, UPDATE and DELETE.

19. Delta transaction log

The _delta_log stores ordered commits and metadata describing the files and state of each table version. It enables consistent snapshots, history, conflict handling and time travel.

20. ACID Properties

Property	Meaning
Atomicity	A transaction commits completely or not at all.
Consistency	Transactions maintain valid table states and rules.
Isolation	Concurrent operations do not expose partial transactions.
Durability	Committed changes persist.

21. Update records and MERGE

```
UPDATE customers
SET city = 'Hyderabad'
WHERE customer_id = 101;


MERGE INTO target t
USING source s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

MERGE is commonly used for upserts: update matching records and insert new records.

22. Can Delta be queried using SQL?

```
SELECT * FROM main.sales.orders WHERE amount > 1000;
```

Part 5 - Large-Scale and Failure Scenarios

23. How would you process 50 million records or 50 GB?

1. Use distributed Spark DataFrames and avoid collecting all records to the driver.

2. Read only required columns; filter early.
3. Inspect partition count and partition sizes.
4. Use suitable join strategies; broadcast only genuinely small data.
5. Check data skew and large shuffles.
6. Use suitable compute and autoscaling/policies where available.
7. Write scalable formats such as Delta/Parquet.
8. Compact small files or cluster data when justified.
9. Validate counts, nulls, duplicates and business rules.

How to say it: I would use Spark DataFrames for distributed processing, filter unnecessary rows and columns early, inspect partitions, optimize joins, check skew and shuffle, use appropriate compute, write to Delta, and validate the results. I would never collect tens of millions of rows to the driver.

24. Production job failed

Investigate → Find root cause → Communicate/escalate → Approved fix → Safe rerun → Validate → Document

10. Open the failed run and identify the failed task.
11. Inspect job, driver and executor logs.
12. Classify the cause: data, code, compute, permissions or dependency.
13. Determine whether partial output was written.
14. Inform the responsible engineer and follow the incident process.
15. Apply an approved fix.
16. Rerun safely, preferably from an idempotent/checkpointed stage.
17. Validate row counts and output quality.

How to say it: As a fresher, I would not make unapproved production changes. I would inspect the failed task and logs, identify whether the problem is data, code, compute, permissions or dependencies, report my findings, apply an approved fix, rerun safely and validate the output.

25. Join failed

- Verify join keys and data types.
- Compare input sizes and inspect the physical plan.
- Check shuffle size and data skew.
- Select only required columns and filter early.
- Consider broadcast for a safe small side.
- Repartition only when it meaningfully improves distribution.
- Inspect executor memory and spill.

26. Insufficient memory

Do not immediately increase the cluster. First inspect the Spark UI and executor logs for skew, huge shuffles, large broadcasts, excessive caching, collect/toPandas calls or oversized partitions.

How to say it: I would inspect Spark UI and executor logs to find the memory pressure. I would check skew, shuffle, partitions, caching, collect operations and join strategy. After optimizing the workload, I would scale compute only if the workload genuinely requires it.

27. Why might a cluster terminate?

- Idle auto-termination
- Job compute completed
- Manual termination
- Cloud/resource or infrastructure issue
- Policy configuration
- Preemptible/spot capacity interruption

How to say it: I would inspect the cluster event log and the job logs. Causes can include auto-termination, completion of job-scoped compute, manual action, resource failure or cloud capacity issues.

Part 6 - Data Quality

28. Remove duplicates in PySpark

```
exact_unique = df.distinct()
key_unique = df.dropDuplicates(["customer_id"])
```

distinct removes duplicate combinations across all columns. dropDuplicates can use selected business-key columns. If duplicate rows contain different values, Spark does not guarantee which surviving row is retained unless you define an ordering rule.

29. Keep the latest or earliest duplicate

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col


w = Window.partitionBy("id").orderBy(col("timestamp").desc())
clean_df = df.withColumn("rn", row_number().over(w)).filter("rn = 1").drop("rn")
```

Use ascending order to keep the earliest record and descending order to keep the latest. Add a unique tie-breaker when timestamps can be identical.

30. Malformed CSV records

18. Define the expected schema.
19. Verify delimiter, quote, escape, encoding and header options.
20. Use a suitable mode such as PERMISSIVE when business rules allow partial ingestion.
21. Capture corrupt records or rejected rows separately.
22. Investigate error reasons and validate quality counts before promotion.

```
df = (spark.read.schema(schema)
    .option("header", True)
    .option("mode", "PERMISSIVE")
    .csv(path))
```

31. Why header=True?

It tells Spark to interpret the first CSV row as column names rather than ordinary data.

Part 7 - Azure Essentials

32. Azure Blob Storage vs ADLS Gen2

Blob Storage	ADLS Gen2
General-purpose Azure object storage.	Azure Storage with data-lake capabilities such as hierarchical namespace and directory-style organization for analytics.

How to say it: Blob Storage is general-purpose object storage. ADLS Gen2 provides data-lake capabilities on Azure Storage, including hierarchical namespace, which is useful for analytics and big-data workloads.

33. Store confidential secrets

Azure Key Vault is used to store and manage secrets, keys and certificates. Do not hardcode passwords, tokens or storage keys in notebooks. Prefer identity-based access such as Microsoft Entra ID and managed identities where supported.

34. SAS token vs storage account key

Storage account key	SAS token
Broad credential with high impact if compromised.	Delegated access that can be limited by resource, permissions and expiry.

Prefer least privilege. Use scoped, short-lived access or managed identities where supported.

35. Managed vs external volumes

Managed volume	External volume
Unity Catalog manages the storage location/lifecycle under managed storage.	References an approved external cloud-storage location while Unity Catalog governs access.

36. Bring on-premises data to Databricks

On-prem SQL → Azure Data Factory + Self-hosted Integration Runtime → ADLS → Databricks → Bronze/Silver/Gold

How to say it: A common Azure approach is ADF with a self-hosted Integration Runtime to copy data securely from an on-premises source to ADLS. Databricks can then process it into Bronze, Silver and Gold layers.

37. Types of sources

Category	Examples
Databases	SQL Server, Oracle, MySQL, PostgreSQL
Files	CSV, JSON, Parquet, Excel
Cloud storage	ADLS, Azure Blob, Amazon S3
APIs	REST APIs
Streams	Kafka, Azure Event Hubs

Part 8 - SQL Interview Core

38. Primary key vs unique key

Primary key	Unique key/constraint
Main row identifier; unique and non-null when enforced.	Enforces uniqueness for another candidate key. NULL rules depend on database engine.
Typically one primary-key definition per table.	Multiple unique constraints may exist.

39. Surrogate key

A surrogate key is an artificial/system-generated identifier such as customer_key = 1001. It has no business meaning and is frequently used in dimension tables.

40. Fact vs Dimension

Fact table	Dimension table
Stores measurable events and foreign keys: quantity, revenue, date_key.	Stores descriptive context: customer name, city, product category.
Usually large and event-focused.	Used to filter, group and describe facts.

41. Normal forms

Form	Main idea
1NF	Atomic values; no repeating groups in a field.
2NF	1NF plus no partial dependency on part of a composite key.
3NF	2NF plus no transitive dependency between non-key attributes.

Normalization reduces redundancy and improves integrity. Denormalization intentionally combines/repeats data to simplify or speed analytical queries.

42. WHERE vs HAVING

```
SELECT department_id, COUNT(*)
FROM employees
WHERE salary > 30000        -- filters rows before grouping
GROUP BY department_id
HAVING COUNT(*) > 5;        -- filters groups after aggregation
```

43. SQL join types

Join	Result
INNER	Matching rows only.
LEFT	All left rows and matching right rows.
RIGHT	All right rows and matching left rows.
FULL OUTER	Matched and unmatched rows from both sides.
CROSS	Every left row paired with every right row.

Warning: FULL OUTER JOIN and UNION ALL are different. FULL OUTER JOIN combines columns horizontally using a condition; UNION ALL appends compatible query results vertically and retains duplicates.

44. Window functions

Salary	ROW_NUMBER	RANK	DENSE_RANK
9000	1	1	1

8000	2	2	2
8000	3	2	2
7000	4	4	3

ROW_NUMBER is unique sequence. RANK gives ties the same rank with gaps. DENSE_RANK gives ties the same rank without gaps. Other useful windows are LAG, LEAD, SUM OVER and AVG OVER.

45. DDL, DML and DCL

Category	Purpose	Commands
DDL	Defines structure	CREATE, ALTER, DROP, TRUNCATE
DML	Changes data	INSERT, UPDATE, DELETE
DCL	Controls access	GRANT, REVOKE

46. ALTER vs UPDATE

```
ALTER TABLE employees ADD email VARCHAR(100);  -- structure
UPDATE employees SET salary = 70000 WHERE employee_id = 10;  -- data
```

47. DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes selected/all rows; WHERE supported.	Removes all rows; table remains.	Removes the table object itself.
DML in common classification.	DDL in common classification.	DDL.

Exact logging, rollback and identity behavior varies by database engine.

48. Nth highest salary

```
WITH ranked AS (
  SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS r
  FROM employees
)
SELECT * FROM ranked WHERE r = 3;
```

DENSE_RANK returns the third distinct salary and all ties. Add PARTITION BY department_id for the nth salary in each department.

49. Find duplicate records

```
SELECT customer_id, COUNT(*) AS copies
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

50. CTE and recursive CTE

A CTE is a temporary named query result defined with WITH and visible within one enclosing statement. A recursive CTE references itself and is useful for hierarchies such as employee-manager trees.

```
WITH high_salary AS (
  SELECT * FROM employees WHERE salary > 50000
)
SELECT * FROM high_salary;
```

51. Subqueries

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- Scalar subquery: returns one value.
- Single-row subquery: returns one row.
- Multi-row subquery: returns multiple rows.
- Correlated subquery: depends on the current outer-query row.

52. Function vs stored procedure

A function usually returns a scalar value or table and may be used in SQL expressions. A stored procedure executes a set of database operations and can contain procedural logic. Exact capabilities vary by database engine.

53. Get first name from full name

Use string functions to find the first space and return the characters before it. Syntax varies by database.

```
-- SQL Server example
SELECT LEFT(full_name, CHARINDEX(' ', full_name + ' ') - 1)
FROM employees;
```

54. LIKE patterns

Pattern	Meaning
'S%'	Starts with S
'%a'	Ends with a
'%mi%'	Contains mi
'_a%'	Second character is a

% means zero or more characters; _ means exactly one character.

55. Constraints

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

56. Indexing

An index is a database data structure that helps locate rows efficiently for supported access paths. It uses storage and adds maintenance overhead to INSERT, UPDATE and DELETE operations.

57. Evaluate query performance

- Inspect the execution plan.
- Look for expensive scans, joins, sorts and aggregations.
- Check indexes, partition pruning and join conditions.
- Avoid SELECT * when only a few columns are required.
- Filter early when logically safe.
- For Spark SQL, inspect Spark UI, shuffle, spills, skew, partitions and broadcast choices.

58. View vs materialized view

View	Materialized view
Stores a query definition; underlying query is evaluated when used.	Stores/precomputes query results and refreshes according to platform rules.
Always reflects underlying data according to query execution.	Can improve repeated expensive queries but may require refresh and storage.

Part 9 - Data Warehousing

59. Data warehouse characteristics

- Subject-oriented: organized around business subjects such as sales or customer.
- Integrated: standardizes data from multiple sources.
- Time-variant: maintains historical data.
- Non-volatile: primarily supports stable analytical reading rather than constant transactional updates.

60. OLTP vs OLAP

OLTP	OLAP
Day-to-day transactions such as orders and payments.	Analytical queries such as annual revenue by region.
Current operational data.	Historical and aggregated data.
Frequent small reads/writes.	Complex, scan-heavy analytical queries.

61. Star vs Snowflake schema

Star	Snowflake
Central fact table directly connected to denormalized dimensions.	Dimensions are further normalized into related tables.
Simpler and usually easier for BI queries.	Reduces dimension redundancy but adds joins and complexity.

62. Measure types

Type	Example	Rule
Additive	Sales amount	Can be summed across all relevant dimensions.
Semi-additive	Account balance	Can be summed across some dimensions, not usually across time.
Non-additive	Percentage or ratio	Should not be directly summed.

63. Dimension types

Type	Meaning/example
Conformed	Shared consistently across multiple fact tables.
Role-playing	One Date dimension used as Order Date, Ship Date and Delivery Date.
Degenerate	Identifier such as invoice_number stored in the fact table without a separate dimension.
Junk	Combines low-cardinality flags and indicators.

64. Slowly Changing Dimensions

SCD	Treatment	History
Type 1	Overwrite old value	No history
Type 2	Insert a new versioned row	Full history
Type 3	Store current and previous values in columns	Limited history

How to say it: Type 1 overwrites and does not preserve history. Type 2 inserts a new row with dates/current flag and preserves history. Type 3 stores limited previous values in additional columns.

Part 10 - Python Fundamentals

65. Data types and collections

Category	Types
Basic	int, float, str, bool, None
Collections	list, tuple, set, dict

66. List vs Tuple vs Dictionary

List	Tuple	Dictionary
Ordered, mutable sequence.	Ordered, immutable sequence.	Key-value mapping.
["Ravi", "Sam"]	(10, 20)	{"id": 101, "name": "Ravi"}

67. Reverse a string

```
name = "Soumiya"
print(name[::-1])   # ayimuoS
```

68. String interpolation

```
name = "Soumiya"
print(f"Hello {name}")
```

69. Shallow vs deep copy

A shallow copy creates a new outer container but may share nested objects. A deep copy recursively copies nested objects.

```
import copy
shallow = copy.copy(original)
deep = copy.deepcopy(original)
```

70. Types of arguments

- Positional: add(10, 20)
- Keyword: add(a=10, b=20)
- Default: def greet(name="Guest")
- Variable positional: *args
- Variable keyword: **kwargs

71. OOP concepts

Concept	Meaning
Encapsulation	Bundle data and methods and expose a controlled interface.
Inheritance	A child class derives/reuses behavior from a parent.
Polymorphism	The same interface can behave differently for different objects.
Abstraction	Expose essential behavior while hiding implementation details.

72. Constructor, self and __init__

```
class Employee:
    def __init__(self, name):
        self.name = name


e = Employee("Soumiya")
```

__init__ initializes an instance. self refers to the current object.

73. Lambda function

```
square = lambda x: x * x
print(square(5))  # 25
```

74. Decorator

A decorator wraps a function or class to add or modify behavior without rewriting its main implementation. Beginner syntax: @decorator_name above a function.

75. Overriding vs overloading

Overriding: a child class provides its own implementation of a parent method. Traditional compile-time overloading by function signature is not directly supported in Python; later definitions replace earlier ones. Similar flexibility is commonly achieved using default arguments, *args or single-dispatch techniques.

76. Connect Python to SQL

Use a database-specific driver or SQLAlchemy. Examples: pyodbc for SQL Server, psycopg for PostgreSQL and mysql-connector-python for MySQL. Spark can use JDBC for distributed database reads/writes. Keep credentials outside code.

Part 11 - HR, Training and Project Answers

77. Why Data Engineering?

How to say it: During my training, I became interested in how raw data is collected, cleaned, transformed and made ready for analytics. I enjoyed working with SQL, PySpark, Databricks and Medallion architecture. Data engineering interests me because it combines programming, data processing and problem solving.

78. What did you learn in two months / LTM?

How to say it: I first strengthened SQL and Python fundamentals. Then I learned Spark and PySpark for distributed processing, including DataFrames, transformations, joins, partitions and windows. I learned Databricks and Delta Lake concepts such as Bronze-Silver-Gold, schema handling and Delta tables. I also learned Azure basics such as ADLS, Blob Storage and ADF. The main improvement was understanding how these components connect into an end-to-end data pipeline.

79. Challenge faced during training

How to say it: One challenge was understanding duplicate handling. Initially I considered only dropDuplicates, but I learned that records with the same business key and different timestamps require a deterministic rule. I used ROW_NUMBER over a window ordered by timestamp, retained the required row and validated the final count.

80. Technical problem using STAR

STAR	What to explain
Situation	Context and problem.
Task	Your responsibility.
Action	Specific steps and tools you used.
Result	Validated outcome or learning.

How to say it: While processing a training dataset, I found duplicate records for the same business key. I identified duplicate keys using groupBy and count. Since the records had timestamps, I used ROW_NUMBER with a window partitioned by the key and ordered by timestamp. I retained row number 1 and validated the counts. This taught me that deduplication requires a business rule.

81. Project explanation structure

23. Problem statement
24. Data sources
25. Architecture and tools
26. Your own responsibility
27. Transformations and quality checks
28. Challenge and resolution
29. Final output and measurable validation

How to say it: The objective of my project was ____. We received data from ____. My contribution was ____. I used ____ to ingest and process it. In Silver I handled nulls, duplicates and data types. In Gold I created ____. One challenge was ____, which I solved by ____. The final validated output was ____.

82. Self-introduction structure

30. Name and education
31. Relevant training
32. Key technical skills
33. One truthful project contribution
34. Strength or learning behavior
35. Entry-level data engineering goal

Keep it around 60-90 seconds and use only truthful project details.

Part 12 - Interview Traps and Rapid Revision

Ten traps to avoid

36. Do not say a DataFrame is only a table. Mention distributed structured data.
37. Do not broadcast solely because the file is 50 MB. Confirm memory and plan.
38. Repartition shuffles data.
39. Coalesce and repartition are not identical.
40. Delta is not the same as Parquet. Delta commonly uses Parquet plus a log.
41. Do not cache everything. Cache reused expensive data.
42. Do not blindly restart a failed production job. Investigate and validate.
43. FULL OUTER JOIN is not UNION ALL.
44. dropDuplicates does not define a deterministic survivor for differing rows.
45. Do not treat every memory issue by immediately increasing the cluster.

One-minute revision sheet

Term	Memory line
Databricks	Cloud data and AI platform.
Spark	Distributed processing engine.
Driver	Coordinates the Spark application.
Executor	Runs tasks on partitions.
DataFrame	Distributed structured rows and columns.
Catalyst	Spark SQL query optimizer.

repartition	Shuffle and redistribute.
coalesce	Usually reduce with less movement.
Broadcast join	Send safe small side to executors.
Bronze/Silver/Gold	Raw / cleaned / business-ready.
Delta	Data files plus transaction log and table features.
Parquet	Columnar file format.
ACID	Atomicity, Consistency, Isolation, Durability.
MERGE	Update matches and insert new records.
WHERE/HAVING	Rows before grouping / groups after aggregation.
ROW_NUMBER/RANK/DENSE_RANK	Unique / ties with gaps / ties without gaps.
OLTP/OLAP	Transactions / analytics.
SCD 1/2/3	Overwrite / new row history / previous column.
ADF	Data integration and orchestration.
ADLS	Azure analytics-oriented data lake storage capabilities.

7-Day Preparation Plan

Day	Focus
1	Spark architecture, Driver/Executor, RDD, DataFrame, transformations/actions, Catalyst.
2	Partitions, repartition, coalesce, shuffle, broadcast, cache/persist, skew and memory.
3	Databricks, compute, notebook, Medallion, Delta, Parquet, Delta log, ACID and MERGE.
4	SQL keys, joins, GROUP BY, WHERE/HAVING, windows, CTE, subqueries and nth salary.
5	OLTP/OLAP, fact/dimension, star/snowflake, measures, dimensions and SCD.
6	ADF, ADLS, Blob, Key Vault, SAS, Python collections, arguments and OOP.
7	Speak answers aloud: introduction, project, why DE, training, failures and optimization.

Final Speaking Formula

For definitions: Definition → Why it matters → Small example.

For scenarios: Investigate → Identify cause → Apply approved fix → Validate.

For projects: Problem → Source → Architecture → Your contribution → Challenge → Result.

Your strongest fresher answer is not pretending to have years of production experience. It is showing that you understand what to check, why to check it, how to work safely, and how to validate the result.